

Week 6: Building Testable Design Programs

Generate candidates, shorten safely and choose an accepted route

School of Mechanical Engineering, Soongsil University
2026-2 Applied Programming

Week 5 measured supplied paths. This week the program must build alternatives, shorten them safely and select one that passes the length and clearance limits. Separating generation, improvement, evaluation and selection lets us check each decision on its own.

Week 2's numerical functions provide calculations and its tests check returned results. We reuse Week 3's `Configuration` and `Path` objects, and Week 4's dictionaries and JSON. The objects own their route data, `path.length()` adds segment lengths, and a dictionary associates each route with an ID. JSON stores the results. The new work is to generate combinations of two parameters and **change the candidate routes before evaluating them**.

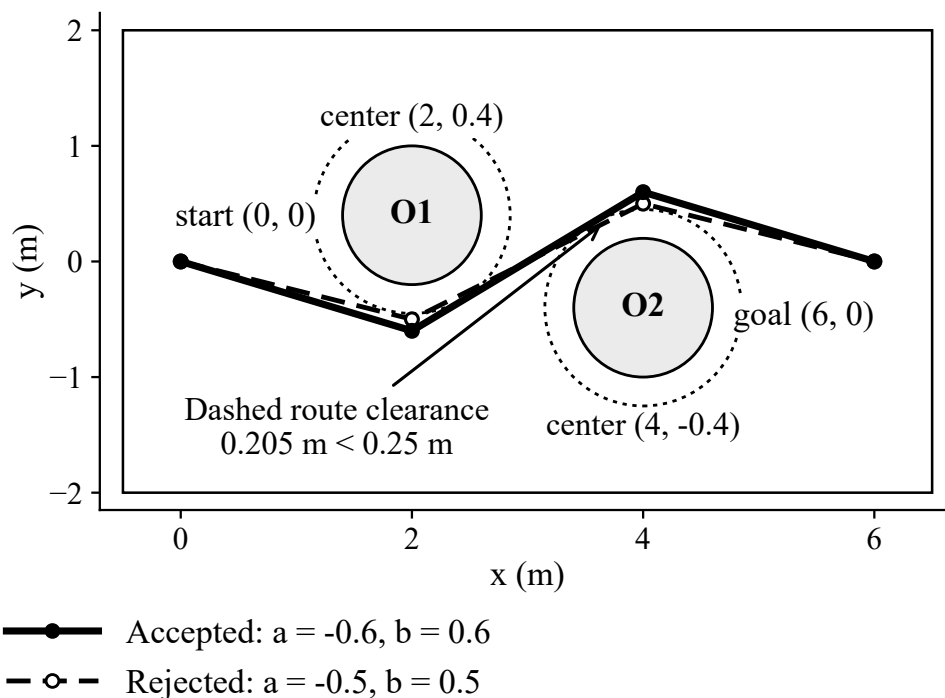
1 Build a route that satisfies two limits

Our point starts at $(0, 0)$ and must reach $(6, 0)$. The workspace allows $-0.5 \leq x \leq 6.5$ and $-2 \leq y \leq 2$. Two circular obstacles have centres $(2, 0.4)$ and $(4, -0.4)$, each with radius 0.6 m. All coordinates and distances are in metres. The first obstacle lies above the straight start–goal line and the second lies below it. Passing below the first and above the second can reduce the required detour. The point represents position, not a robot body. Every segment must remain at least 0.25 m from both disk surfaces, and the final route must be no longer than 6.8 m.

We choose two values: the first bend height a near the first obstacle and the second bend height b near the second. They can differ. The supplied constructor builds this initial route:

```
(0, 0) -> (0, a) -> (2, a) -> (4, b) -> (6, b) -> (6, 0)
```

When a or b is zero, the constructor removes consecutive duplicate points. Start, goal, obstacles, bounds and limits stay fixed. Only the bend values and the points removed by our shortening procedure vary.



Solid circles: obstacles (radius 0.60 m).

Dotted circles: required clearance boundary (radius 0.85 m).

Figure 6.1: Two staggered obstacles require two independently chosen bend heights. The solid route passes both limits after shortening. The slightly shorter dashed route passes too close to an obstacle. The circles show the obstacles, not a robot's shape.

An **objective** tells us which accepted result to prefer: here, the smallest length. A **constraint** decides whether a candidate may enter that comparison: here, clearance, length and workspace bounds.

The dotted circles show obstacle radius plus required clearance, $0.6 + 0.25 = 0.85$ m. The dashed route avoids the solid disks but enters that clearance region along its middle segment. Its stored bend points alone look acceptable. The segment between them is what fails.

For example, the shortened $a = -0.5, b = 0.5$ route has length about 6.359 m, but its minimum clearance is only 0.205 m. The $a = -0.6, b = 0.6$ route is longer, about 6.509 m, and keeps about 0.257 m clearance. Increasing the detour to $a = -0.8, b = 0.8$ raises clearance to about 0.337 m but length to about 6.869 m. Only the middle choice passes both limits. These displayed decimals are rounded for reading. The program compares the stored values.

1.1 Prepare one workspace for practice and the Assignment

Use the Week 6 template link announced through LMS and the GitHub account and authentication procedure taught in Weeks 1–4. Select **Use this template** and **Create a new repository**. Under your own account, create a **Private** repository named `applied-programming-w06-YOUR-STUDENT-ID`. Replace `YOUR-STUDENT-ID` with your student ID. Keep your earlier repositories. Use this same Week 6 repository for practice and the Assignment.

Copy its HTTPS clone URL with the **Code** button. Open **Anaconda Prompt**, not Python's `>>>` prompt. The folder and clone URL below are replacement values.

Run – your course parent folder

```
cd /d "C:\your-course-folder"
git clone YOUR-PRIVATE-HTTPS-CLONE-URL
cd applied-programming-w06-YOUR-STUDENT-ID
git remote -v
```

Both `origin` addresses must name your own Week 6 repository. If sign-in is requested, use the previously taught Git Credential Manager procedure. Never place a password or token in the URL.

The environment keeps Week 5's versions under the new name `applied-programming-w06`. The supplied `environment.yml` selects Python 3.12.13 and pip 26.2.1. The requirements select NumPy 2.4.6, pytest 9.1.1 and setuptools 80.9.0. No GPU or simulator is needed.

Run once – Anaconda Prompt, repository root

If this Week 6 environment already exists, continue with activation instead of recreating it. Keep saved work and existing environments.

```
set PYTHONUTF8=1
conda --no-plugins env create --solver classic --file environment.yml
```

Activate, install and check – same folder

```
conda activate applied-programming-w06
python -m pip install -r requirements.txt
python -m pip install -e . --no-build-isolation
python scripts/verify_environment.py
```

Expected status

```
ENVIRONMENT PASS
```

The checker verifies versions and imports, not student TODOs. Resolve any reported mismatch before running examples. Installation needs network access. Examples and tests then run locally. When reopening Anaconda Prompt, return to the repository root, set `PYTHONUTF8` and activate this same environment.

Locate the complete examples before running them

These files run before the Assignment functions are completed:

- `examples/inspect_routes.py` evaluates supplied initial and shortened routes. It prints their lengths and acceptance results.
- `examples/inspect_clearance.py` compares point sampling and whole-segment clearance on the same Week 5 path.

The complete examples do not call your unfinished generation or shortening functions. They let you inspect the inputs and expected meanings first. The final application and grid comparison use your implementations and are run after the relevant functions are completed. No additional environment or simulator is needed for either stage.

2 Separate the route from the way it is checked

First create or shorten a route, then evaluate that exact path. Keeping these steps separate ensures that its reported length describes its stored coordinates.

A **module** is a Python file that provides definitions to other code. Here `routes.py` supplies `RouteProblem`, `RouteSettings` and the evaluator for the two-obstacle scene and whole-segment checking. Week 5's supplied

`ParametricPlanningProblem` remains unchanged for its one-obstacle, sampled-measurement examples. Your three functions in `selection.py` generate and shorten routes, then select an ID. That module also contains the completed evaluation function.

2.1 Interpolation checks a path, rather than inventing another one

Interpolation here means calculating positions along an existing straight segment. In Week 5, `waypoints` checked the stored points, whereas `interpolated` added measurement points between them. These extra points change where distances are measured, not the route's shape or length.

A sampled minimum can miss the closest position. To see the consequence, keep the Week 5 disk at (2, 0) with radius 0.5 m. Consider $(0, 0) \rightarrow (2, 1.5) \rightarrow (4, 0)$. Its length is 5 m. With interpolation spacing 0.5 m, the smallest measured clearance is about 0.704159 m. The smallest clearance over the entire segments is 0.700000 m. If the required clearance is 0.703 m, these two checks disagree.

The supplied `continuous_segments` calculation finds the position on each segment closest to each obstacle centre, then subtracts the obstacle radius from that distance. The smallest result over all segment–obstacle pairs is the route clearance. This calculation applies to a point following straight segments around fixed circular obstacles.

This table answers whether the measurement method can change the chosen route. Both alternatives use minimum clearance 0.703 m and maximum length 6 m. “Sampled” uses the same 0.5 m spacing for both.

Bend height (m)	Length (m)	Sampled clearance (m)	Whole-segment clearance (m)
1.5	5.000000	0.704159: pass	0.700000: fail
1.6	5.122499	0.757864: pass	0.749390: pass

The sampled check selects the shorter, 1.5 m bend route. The whole-segment check rejects it and selects the 1.6 m bend route. Both routes avoid the disk, but only the second keeps the required 0.703 m clearance along the entire path.

Run – repository root, before Assignment TODOs

```
python examples/inspect_clearance.py
```

Expected output

```
height_m=1.5, sampled_clearance_m=0.704159458, continuous_clearance_m=0.700000000
minimum=0.703: sampled_pass=True, continuous_pass=False
height_m=1.6, sampled_clearance_m=0.757864151, continuous_clearance_m=0.749390095
minimum=0.703: sampled_pass=True, continuous_pass=True
```

The completed file evaluates these same two paths with both measurement methods. The new two-obstacle task uses whole-segment checking throughout.

2.2 Use one evaluation request and a precise answer

An **interface** specifies an agreed request and answer. The code calls `evaluator.evaluate(problem)` and receives a report. Code that makes this request is the **caller**. It uses the reported acceptance and length without performing geometry itself.

The supplied `RouteEvaluator` in `routes.py` accepts a `RouteProblem` and returns an `EvaluationReport`. Selection reads `report.valid` to check acceptance and `report.metrics.length_m` to compare lengths. Within the metrics, **feasible** means the length and clearance margins are nonnegative. **valid** also checks the complete route against the supplied scene constraints. These fields describe checking, not execution.

For this whole-segment method, `sample_count=0` means no interpolated measurement points were created, even though all segments were checked.

Open the completed `examples/inspect_routes.py`. `raw = problem_at_bends(-0.6, 0.6)` creates the six-point initial problem. `final` names the four-point comparison problem with coordinates (0,0), (2, -0.6), (4, 0.6) and (6, 0). `evaluator = RouteEvaluator()` creates the checking object.

Code – actual calls in `examples/inspect_routes.py`

```
reports = {}
for label, problem in (("raw", raw), ("final", final)):
    report = evaluator.evaluate(problem)
    reports[f"{label}_report"] = asdict(report)
    print(f"{label}: length_m={report.metrics.length_m:.6f}, "
          f"clearance_m={report.metrics.clearance_m:.6f}, valid={report.valid}")
```

The evaluation call returns a report, assigned to `report`. Reading `report.metrics.length_m` obtains its length, and `print` displays it. The supplied `asdict(report)` converts the report's fields into a dictionary for JSON storage.

Run – repository root, before Assignment TODOs

```
python examples/inspect_routes.py
```

Expected output

```
raw: length_m=7.532381, clearance_m=0.257493, valid=False
final: length_m=6.508503, clearance_m=0.257493, valid=True
outer shortcut allowed: True
inner shortcut allowed: False
saved=artifacts/route_inspection.json
```

The initial route fails the length limit, while the shorter route passes both limits. The example also checks an allowed outer shortcut and a rejected middle shortcut.

2.3 Ask whether a new connection is safe

Before removing a corner, the shortener needs to know whether the new direct connection is allowed. In the completed `examples/inspect_routes.py`, `edge_checker_for(raw)` returns a checking function for that problem's workspace, obstacles and required clearance. The name `check` refers to this returned function.

Code – checking calls in `examples/inspect_routes.py`

```
check = edge_checker_for(raw)
print("outer shortcut allowed:", check(raw.path.waypoints[0], raw.path.waypoints[2]))
print("inner shortcut allowed:", check(final.path.waypoints[0], final.path.waypoints[2]))
```

The first check accepts the outer shortcut from (0,0) to (2, -0.6). The second rejects the middle shortcut from (0,0) to (4, 0.6). Each check takes two `Configuration` objects and returns `True` or `False`, giving the two Boolean lines in the preceding output.

In `shorten_route(path, edge_checker)`, `edge_checker` is the parameter that receives such a supplied function. Passing `check` gives the function itself to the shortener. Calling `edge_checker(left, right)` inside your completed shortener runs that function for a proposed connection. You implement which point to remove, while the supplied function checks whether the direct connection stays inside the workspace and keeps clearance.

3 Generate and safely shorten the candidates

We first generate routes by choosing two bend heights, then shorten each route using the supplied connection checker.

3.1 Two grid axes produce combinations, not paired heights

A **grid search** tries regularly spaced parameter values. Every value of a combines with every value of b . Pairing only matching indices would try $a = b$ and miss routes that pass below one obstacle and above the other. The axes are bend heights, not the route's x positions.

Week 2's `np.linspace(low, high, count)` includes both endpoints. For $[-1, 1]$ with count 3, each axis contains $[-1.0, 0.0, 1.0]$. There are $3 \times 3 = 9$ combinations. With a in the outer loop and b in the inner loop, IDs begin `grid_0_0`, `grid_0_1`, `grid_0_2`, `grid_1_0`. The supplied `problem_at_bends(a, b, settings)` constructs each initial `RouteProblem`. Your generator returns their new ID-to-problem dictionary.

What does a finer grid change? This comparison keeps the scene, limits, checker and shortening rule fixed. Both axes cover $[-1.6, 1.6]$ m.

Count per axis	Candidates	Accepted after shortening	Selected result
9	81	0	None
17	289	1	<code>grid_5_11</code> , about 6.509 m

Spacing changes from 0.4 to 0.2 m. The finer grid includes $a \approx -0.6$, $b \approx 0.6$, which the coarse grid misses. Finding none in 81 trials does not prove no route exists. The winner is shortest among these generated and improved candidates, not among every possible route. Grid refinement changes candidate routes. Interpolation adds measurement points to an existing route.

3.2 Shortening changes the actual route

For $a = -0.6$, $b = 0.6$, the initial six-point route is about 7.532 m long, above the 6.8 m limit. Removing its two outer corners replaces each two-segment turn with one safe diagonal. The four-point result is about 6.509 m long.

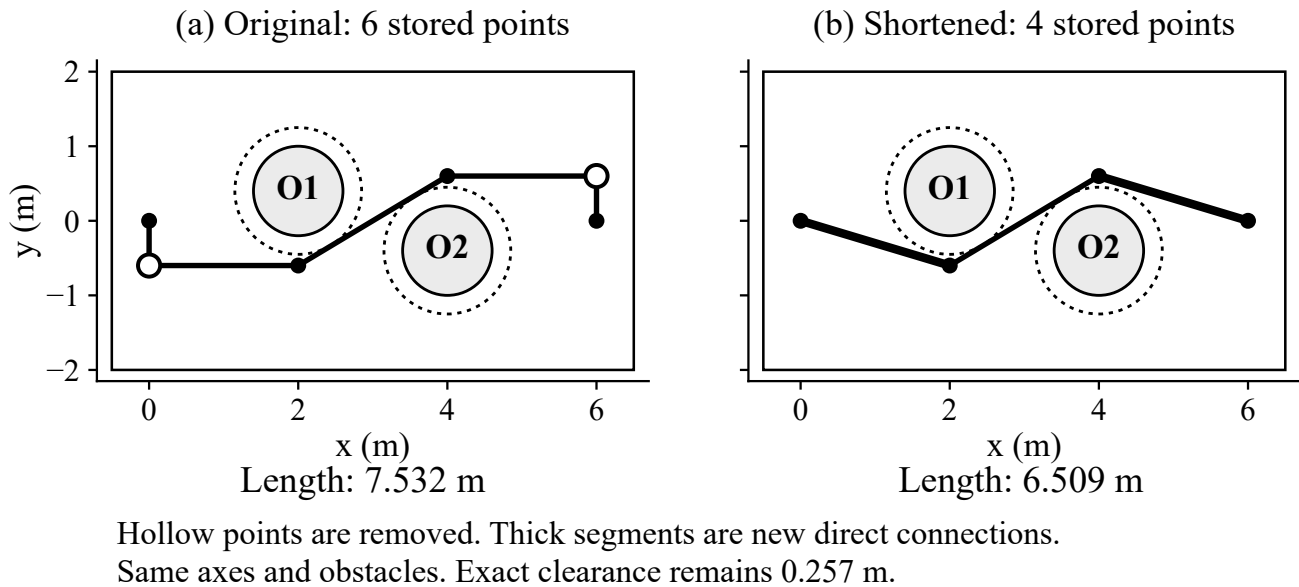


Figure 6.2: The same candidate before and after safe shortening. The two outer corners are removed. The middle bends remain because their direct replacements violate clearance. Both panels use the same axes, obstacles and limits.

A **shortcut** removes an internal point and directly connects its neighbours. Its saving is the two old segment lengths minus the new length. The supplied checker must also approve that new connection.

Which point should we remove first? For this same initial route, removing $(0, -0.6)$ saves $0.6 + 2 - \sqrt{2^2 + 0.6^2} \approx 0.512$ m. The table compares all four possible deletions in the first pass. Values are rounded in metres.

Remove point	New connection	Saving (m)	Safe?
$(0, -0.6)$	$(0, 0) \rightarrow (2, -0.6)$	0.512	Yes
$(2, -0.6)$	$(0, -0.6) \rightarrow (4, 0.6)$	0.156	No
$(4, 0.6)$	$(2, -0.6) \rightarrow (6, 0.6)$	0.156	No
$(6, 0.6)$	$(4, 0.6) \rightarrow (6, 0)$	0.512	Yes

The two safe deletions save exactly the same length. Our tie rule removes the first of these points in the current path sequence, $(0, -0.6)$, first.

After deletion, compare again. The neighbours of $(2, -0.6)$ are now $(0, 0)$ and $(4, 0.6)$. Its proposed saving changes from 0.156 to 0.376 m, but the new connection is still unsafe. The second pass removes $(6, 0.6)$. Neither remaining internal point can then be removed safely, so the algorithm stops.

At each pass, choose the safe deletion with the largest saving strictly above 10^{-12} m. On an exact tie, keep the first qualifying internal point in the current path sequence. Repeating useful small changes to the current route is **local improvement**. Our rule is **greedy**: choose the best permitted change now, apply it, then repeat. It does not examine every deletion sequence.

Deletion order can matter. With $a = 1.2$, $b = 1.4$ and a 7.8 m length limit, greatest-saving deletion gives 7.646 m, while first-safe deletion gives 8.603 m. Both keep clearance, but only the former meets that limit. Greedy shortening is not guaranteed to find the best route in every problem.

Algorithm – one repeated improvement step

```

copy the path's points into a working list
repeat:
    find the safe one-point deletion with largest saving > 1e-12 m
    if no deletion qualifies: stop
    remove that one point from the working list
return a new Path built from the remaining points

```

This is an algorithm description, not executable Python. Start and goal are never removed. Each deletion reduces the point count, so an n -point path permits at most $n - 2$ deletions.

Improve before applying the final length limit. Rejecting the initial 7.532 m route would lose its accepted 6.509 m result. A shortcut must be safe and save length. The final evaluator checks the whole resulting route, including any unsafe segments left unchanged.

Path owns an immutable sequence of Configuration objects. Modify a new list made from `path.waypoints`, then return a new Path. The original coordinates remain available for comparison.

Later lessons add random generation and parameter changes under an evaluation budget. Graph and sampling planners also reuse the idea of checking a proposed connection before accepting it.

4 Evaluate, select and preserve the actual result

The complete program generates candidates, shortens their routes, evaluates the final paths and selects an accepted result.

The supplied `prepare_routes` helper puts each shortened path in a new problem under the same candidate ID. These caller lines are in `workflow.py` and run after your three functions are complete.

Code – stage connection in workflow.py

```

prepared, shortening = prepare_routes(candidates)
results = evaluate_candidates(prepared, evaluator)
selected_id = select_shortest(results)

```

`candidates` holds the initial problems. `prepared` holds the new problems with shortened paths. `shortening` keeps their before/after coordinates. Evaluation, selection, saving and replay use `prepared`, so every reported result refers to the same final route.

4.1 Use the provided evaluations to select an accepted ID

The supplied `evaluate_candidates(candidates, evaluator)` returns a dictionary `results` under the same candidate IDs. Each value is a supplied `CandidateResult` object from `results.py`. Its `report` attribute holds the returned `EvaluationReport`, or `None` when no report is available. Its `error` attribute holds an error message, or `None` when the evaluation completed without an error. A completed evaluation can still reject a route with `report.valid=False`.

Your `select_shortest(results)` ignores errors, missing reports and `report.valid=False`. Among the accepted reports, return the ID with the smallest `report.metrics.length_m`. Keep the first on an exact length tie. Return `None` if none is accepted.

How does the choice change as we read the results? The completed selection test in `tests/test_student.py` compares two shortened detours and a blocked straight route. It uses a **6.9 m length limit** so that both detours pass. The main application still uses **6.8 m**. Clearance remains at least 0.25 m. Read these records from top to bottom.

ID	Length (m)	Clearance (m)	Accepted?	Best ID so far
long	6.869	0.337	Yes	long
blocked	6.000	-0.200	No	long
short	6.509	0.257	Yes	short

The shortest row, `blocked`, cannot win because it fails clearance. `short` replaces `long` because both pass and it is shorter.

For the `short` row, read the dictionary entry and then its report:

Code – illustration using the short record above

```
record = results["short"]
report = record.report
```

Here `record.error` is `None`, `report.valid` is `True`, and `report.metrics.length_m` is about 6.509 m. The dictionary lookup obtains the record. Attribute access then reaches the report and its measurements. These lines read values and do not print them.

This is **best-so-far selection**. Start with no accepted ID, skip rejected records and update only when an accepted route is shorter. It combines Week 2's loop and return values with Week 4's dictionaries.

4.2 Run the improved grid search

After completing the three functions in Section 5, run the supplied main program. It uses 17 values on each axis.

Run – repository root, after TODOs 1–3

```
python -m ap_week06
```

Expected output

```
candidate_count=289
accepted_count=1
selected_id=grid_5_11
a_m=-0.600, b_m=0.600
raw_length_m=7.532381
selected_length_m=6.508503
clearance_m=0.257493
measurement_method=continuous_segments
sample_count=0, segment_count=3
completed=True
final=[6.0, 0.0]
error_count=0
comparison_complete=True
saved=artifacts/candidate_comparison.json
```

`grid_5_11` has $a \approx -0.6$, $b \approx 0.6$. Its length changes from 7.532 to 6.509 m, and its clearance is about 0.257 m. Both main limits pass. The supplied geometric replay follows the final path to `[6.0, 0.0]` without simulating dynamics.

Now run the supplied grid comparison with the same scene, checker and limits.

Run – repository root, after TODOs 1–3

```
python examples/compare_grids.py
```

Expected output

```
count_per_axis=9, candidate_count=81, accepted_count=0, selected_id=None  
count_per_axis=17, candidate_count=289, accepted_count=1, selected_id=grid_5_11  
selected_length_m=6.508503
```

The 9×9 grid misses the accepted route and returns `None`. The 17×17 grid finds it. Only the available candidates change.

Optional view – repository root, after the application succeeds

The main program saves `artifacts/candidate_comparison.json`. The supplied renderer reads it and draws the selected route before and after shortening, showing the removed corners.

```
python scripts/render_selection.py
```

Expected output

```
saved=artifacts/candidate_selection.svg
```

Open `artifacts/candidate_selection.svg` in a web browser. Saving, drawing and replay are complete supplied code.

5 Assignment

5.1 Purpose and work to complete

Build a program that generates alternative routes, removes unnecessary corners safely and chooses the shortest route that passes the limits. Use the same two-obstacle scene from Section 1.

Edit the three marked bodies in `src/ap_week06/selection.py`:

1. `generate_route_grid`: generate every combination of two bend heights.
2. `shorten_route`: repeatedly remove the best safe internal point.
3. `select_shortest`: choose the shortest accepted candidate.

The evaluation function, geometric checker, test files and output code are complete. Keep their supplied code unchanged. Your required edit is `selection.py`. The tests are provided for you to run.

Work in this order

1. Reuse the repository and environment prepared in Section 1.1.
2. Complete the three marked functions.
3. Run the provided tests and fix any failed implementation.
4. Run the main program and the grid comparison from Section 4.2.
5. Push your implementation and submit its repository URL and full commit ID.

5.2 Prepare and recognise the starter baseline

Reuse your `applied-programming-w06-YOUR-STUDENT-ID` repository and `applied-programming-w06` environment. The supplied inspection examples run before your functions are completed. The main program needs those completed functions.

Run – untouched starter, repository root

```
python -m pytest tests/public tests/test_student.py -q
```

Expected summary

```
42 failed, 21 passed
```

The unfinished bodies raise `NotImplementedError`. Their test failures are expected at this stage. An import or environment error is a different problem. Execution time can vary between computers.

5.3 Implement the three marked functions

The functions return values for the supplied program. They do not print results or write files.

TODO 1 – Generate every two-height combination

Function: `generate_route_grid(height_min_m, height_max_m, count, settings)`. Call the supplied `validate_route_grid_parameters` first. It returns `low`, `high` and `count` after checking the inputs. Use `np.linspace(low, high, count)` for both *a* and *b*. Visit *a* in the outer loop and *b* in the inner loop.

For indices *i*, *j*, call `problem_at_bends(float(a), float(b), settings)` and store the result under `f"grid_{i}_{j}"`. Return the new dictionary of all *count*² combinations. For example, three values on each axis must give nine candidates, including pairs where *a* and *b* differ. The supplied helper supports a range inside `[-1.6, 1.6]` and count 2–25. The supplied `problem_at_bends` constructor handles duplicate points caused by a zero height.

TODO 2 – Repeatedly remove the best safe internal point

Function: `shorten_route(path, edge_checker)`. Call `validate_shortening_inputs(path, edge_checker)` to obtain a new working list. For each internal point, compare its two neighbouring segment lengths with the proposed direct segment. Use the supplied `segment_length(left, right)` for each distance.

Consider a deletion only when the saving is greater than 10^{-12} m and `checked_edge(edge_checker, left, right)` returns `True`. Choose the greatest saving. On an exact tie, choose the first qualifying internal point in the current path sequence. Delete that one point and restart on the changed sequence. Stop when no point can be removed this way.

Return `Path(tuple(working_points))`. Preserve start, goal and the original path. The 7.532 m initial route is allowed into this shortening step: the final length limit is checked after improvement. Section 3.2 gives the six-to-four example and explains why the two middle bends must remain.

TODO 3 – Select the shortest accepted route

Function: `select_shortest(results)`. The provided evaluation function has already checked each candidate. Skip records with an error, a missing report or `report.valid=False`. Compare `report.metrics.length_m` for the accepted reports. Return the shortest candidate's ID. Keep the first on an exact tie. Return `None` when none is accepted. Use the existing reports instead of evaluating routes again.

5.4 Run the provided checks

The repository contains complete tests. Run them after implementing the three functions. A failed test identifies behaviour to correct in your implementation. You do not need to design additional tests for this Assignment.

Run – completed implementation, repository root

```
python -m pytest tests/public tests/test_student.py -q
python -m ap_week06
python examples/compare_grids.py
python scripts/verify_submission.py
```

Expected test and checker summaries

```
63 passed
LOCAL CHECK PASS
```

Check the two core results: the fine grid selects `grid_5_11`, whose route shortens from about 7.532 m to 6.509 m, and the coarse grid returns `None`. The saved JSON and optional drawing are generated by the provided code. No separate record, test report or output-file implementation is required.

Completion conditions

- The three functions satisfy their stated behaviour.
- The provided tests and local checker pass.
- Main and grid-comparison runs reproduce the stated core results.
- Your implementation is pushed and identified by the submitted full commit ID.

5.5 Push the code and submit through LMS

Use the same private Week 6 repository and invite SSUMechE through its access settings. A timely pending invitation to this exact account is not a missing submission. Check private visibility and access separately. LMS does not require an additional confirmation sentence.

Inspect your changes and stage the implementation file.

Run – repository root

```
git status --short
git diff
git add src/ap_week06/selection.py
git diff --staged
git commit -m "Complete Week 6 route improvement"
git push origin main
git status --short
git rev-parse HEAD
git ls-remote origin refs/heads/main
```

After pushing, `git status --short` should print nothing. Match the full 40-character `git rev-parse HEAD` result to the first `git ls-remote` value. Open that revision on GitHub and check your implementation file.

Submit these two items through LMS:

- Your own Week 6 repository URL.
- The full 40-character commit ID of the pushed revision to evaluate.

This is a **fictional format example**. Replace both values with your own.

```
Repository URL:
https://github.com/student-example/applied-programming-w06-20261234
Commit ID:
0123456789abcdef0123456789abcdef01234567
```

Use your URL and full pushed commit ID, not the public template URL, a local path, a branch name, `latest`, a short hash or an unpushed commit. Never submit credentials.

Submit within one week of the lab, by Wednesday 11:59 PM on the Week 6 date announced in LMS. GitHub upload is not LMS submission. Later pushes do not change the submitted revision. Missing mandatory requirements at the deadline invoke the announced 0-point submission rule. The timely pending-invitation exception still applies.